

THE INFINITY HACK '26

Simplified Student Challenge Pack

AI Project Manager - Meeting to Execution

Team size: 4 participants | Build time: 3 hours

1. Problem Statement

Build a simple Project Management CRM for NovaWorks Technologies. Provide simple login, a team directory, multiple projects, and project/task screens. Inside the CRM, the administrator pastes a meeting description or transcript. AI automatically creates the projects, identifies tasks, assigns the supplied managers and developers, and sets deadlines and estimated hours.

Goal: Simple login -> Paste meeting -> Create projects and tasks automatically -> View saved projects and assigned tasks.

The focus is the working meeting-to-project flow and a clear frontend. Cost calculation, progress monitoring, user-management screens, signup, and forgot password are not required. Role-based access must limit each logged-in user to their permitted projects and tasks.

2. Company Scenario

Company: NovaWorks Technologies, Lahore, Pakistan.

Business: Websites, mobile apps, and AI tools for clients.

The company has one administrator, three project managers, and six developer agents. It has a meeting about three client projects. Instead of manually entering project and task details, the administrator should paste the supplied transcript into the CRM and create the work automatically.

Simple user-specific screens: the administrator sees all projects and the transcript option; a manager sees projects assigned to them; an agent sees their assigned tasks. Use the logged-in account to control access to projects and tasks. Enforce the same access restrictions in data requests, not only by hiding frontend buttons.

All names and dialogue are fictional. Meeting date: 7 October 2026, 09:00-10:00, Asia/Karachi. All deadlines refer to 2026.

3. Ready-Made Demo Accounts

Use a seeder/setup script to insert the supplied ten demo users, emails, roles, skills, and passwords before the demo. Hardcoded demo credentials and team entries are allowed. Teams may use a setup/seed script to create all ten accounts with passwords, or use a fixed demo-account configuration. Nobody needs to sign up individually. No registration, password reset, email verification, or user-management module is required.

The following credentials are fictional demo values only. Teams may change them and include the final credentials in their README.

Reference	Name / demo email	Role / specialization	Skills	Password
ADMIN	Admin / admin@novaworks.example	Administrator	Company overview, transcript creation	Demo123!
PM01	Ayesha Khan / ayesha@novaworks.example	Manager / Web PM	Web projects, client coordination	Demo123!
PM02	Bilal Ahmed / bilal@novaworks.example	Manager / Mobile PM	Mobile projects, delivery planning	Demo123!
PM03	Hina Malik / hina@novaworks.example	Manager / AI PM	AI projects, requirement review	Demo123!
DEV01	Ali Raza / ali@novaworks.example	Agent / Full-Stack	React, frontend integration	Demo123!
DEV02	Hamza Shah / hamza@novaworks.example	Agent / Full-Stack	Node.js, databases, APIs	Demo123!
DEV03	Sara Noor / sara@novaworks.example	Agent / App Developer	Flutter, mobile UI	Demo123!
DEV04	Usman Tariq / usman@novaworks.example	Agent / App Developer	Flutter, integration, testing	Demo123!
DEV05	Zain Abbas / zain@novaworks.example	Agent / AI Developer	LLMs, extraction, prompts	Demo123!
DEV06	Maryam Asif / maryam@novaworks.example	Agent / AI Developer	Retrieval, document processing	Demo123!

This is ten demo accounts: one admin, three managers, and six agents. These emails are temporary fictional login identifiers, not working inboxes; no verification or email sending is needed. The sample password for every account is Demo123!. These fictional employees are separate from the four hackathon participants. Team/account data and frontend layouts may be hardcoded. The supplied transcript's generated project/task result must still come from the AI flow: a prefilled answer alone does not demonstrate transcript conversion.

Seeder Setup

```
seedDemoUsers():
  for account in suppliedTenAccounts:
    insert account if its demo email does not already exist
    set name, email, role, specialization, skills
    set demo password to Demo123!
// Run before judging. Re-running must not duplicate users.
// No signup, email verification, or password reset needed.
```

4. Minimum Screens and Features

1. Simple login/logout using the supplied demo accounts.
2. Admin home: all project cards and Create from Transcript.
3. Read-only team directory showing names and specializations.
4. Projects list and project detail showing client, manager, deadline, and tasks.
5. Task rows showing title, description, assigned agent, deadline, and estimated hours.
6. Manager view filtered to their projects; agent My Tasks view filtered to their assignments.
7. Persistent saved projects/tasks. A simple local or hosted database is acceptable for this MVP.

Admin pastes the meeting and clicks Create from Transcript. AI processes it with the supplied directory, and the application creates actual records automatically. Show a loading state, success result, or understandable error. For unclear required information, request a correction. Editing created projects/tasks is useful but optional.

5. Simple Requirements

Build a simple CRM with login and role-based access. Admin sees all projects and can create projects/tasks from a transcript. Managers see only their assigned projects. Agents see only their assigned tasks and related projects. Other users' work must not be accessible.

Include multiple projects, task assignments, deadlines, estimated hours, AI transcript conversion, and saved records. Demo accounts may be inserted through a seeder with the supplied temporary emails and passwords.

No signup, forgot password, user-management screens, cost calculation, or progress monitoring is needed. Use any stack; a working local demo is acceptable.

Submission & Deployment

Local database: submit a recorded demo video showing login, transcript conversion, and created projects/tasks.

For extra marks, deploy the application with a hosted database and submit a working live link with demo credentials. Aiven offers a free PostgreSQL database: <https://aiven.io/free-postgresql-database>.

6. Suggested Schema and Pseudocode

Use three main entities: User, Project, and Task. Seed the supplied users; generate projects/tasks from the transcript. IDs below may be strings or database-generated IDs, provided references remain consistent.

Data Schema

```
User {
  id: unique ID
  name: required text
  email: required unique text
  passwordHash: securely stored credential
  role: ADMIN | MANAGER | AGENT
  specialization: text
  skills: list of text
}
Project {
  id: unique generated ID
  name: required text
  clientName: required text
```

```

description: text
managerId: User.id      // must reference a MANAGER
deadline: date          // YYYY-MM-DD
}
Task {
  id: unique generated ID
  projectId: Project.id  // must reference its parent project
  title: required text
  description: text
  assigneeId: User.id    // must reference an AGENT
  deadline: date         // YYYY-MM-DD
  estimatedHours: positive number
}

```

Relationships: one manager can have many projects; one project has many tasks; one agent can have many tasks across projects. Use the same seeded user IDs in the AI directory and saved assignments. Passwords are login credentials and must not be sent to the AI. No cost, hourly-rate, or progress fields are needed.

Seed and Simple Login

```

seedUsers():
  for account in suppliedDemoAccounts:
    upsert by unique email
    save name, role, specialization, skills
    hash sample password Demo123! before storing
login(email, password):
  user = find user by email
  if missing or password does not match:
    return invalid-credentials error
  create login session identifying this user
  open home for user.role

```

Use your stack's standard login/session mechanism. Seed accounts to save time; signup and password-reset screens remain unnecessary. Determine the current user from the login session, not a caller-supplied role or user ID.

User-Specific Access

```

getProjects(currentUser):
  ADMIN -> all projects
  MANAGER -> projects with managerId == currentUser.id
  AGENT -> distinct projects containing their assigned tasks
getTasks(currentUser, projectId):
  ADMIN -> all tasks in projectId
  MANAGER -> all tasks only if they manage projectId
  AGENT -> only tasks assigned to them in projectId
getProjectById(currentUser, projectId):
  reject if project is outside getProjects(currentUser)
  return project details with getTasks(currentUser, projectId)

```

Apply these rules to direct data requests as well as screen lists. An agent may see the related project name and manager, but never other agents' tasks. Only the admin has transcript creation access.

Transcript Automation

```

createFromTranscript(session, transcript):
  user = currentUser(session)
  reject unless user.role == ADMIN
  reject empty transcript
  directory = users with id, name, role, skills
  draft = AI(transcript, directory, expectedOutputShape)
  parse and validate complete draft before saving
  for each project:
    require name, client, manager, valid project date
    require managerId belongs to an existing MANAGER
    for each task:
      require title, existing AGENT, positive hours
      require valid task date <= project deadline
  if invalid:

```

```
    show unresolved fields; save nothing
    allow correction and revalidation
else:
    save projects and linked tasks together
    generate application IDs; keep returned user references
    show created projects and task counts
```

The AI must follow final agreed decisions, ignore rejected features, and use existing users. Disable repeated clicks while processing to avoid accidental duplicates. AI failure or invalid output must not leave partly created projects. Use a database transaction or equivalent all-or-nothing save.

AI Output Shape

```
{
  "projects": [
    {
      "name": "Extracted project name",
      "clientName": "Extracted client",
      "description": "Extracted scope",
      "managerId": "PM01",
      "deadline": "2026-10-20",
      "tasks": [
        {
          "title": "Extracted task",
          "description": "Extracted task scope",
          "assigneeId": "DEV01",
          "deadline": "2026-10-12",
          "estimatedHours": 12
        }
      ]
    }
  ]
}
```

This is a shape example, not a complete answer to seed. The model returns task content and existing user references; application code generates project/task IDs and saves their relationships.

7. Supplied Meeting Transcript

Meeting: NovaWorks Client Delivery Planning

Date: 7 October 2026 | Scheduled duration: 60 minutes

Participants: Ayesha, Bilal, Hina, Ali, Hamza, Sara, Usman, Zain, Maryam

This is a simulated, condensed transcript of a 60-minute meeting. Timestamps represent discussion segments; pauses, repeated explanations, and screen-sharing time are omitted. It is not an hour of continuous verbatim speech. Use the entire transcript as the AI input.

09:00-09:04 | Opening and company workflow

Ayesha: Good morning. We have three client engagements to plan today: UrbanCart Clothing's website, QuickServe's customer mobile app, and HelpDeskPro's AI support assistant. Please keep these as three separate projects. A combined project would make client reporting confusing.

Bilal: We should finish with a project manager, deadline, task owner, and estimated hours for every piece of work. The estimate is effort, not the number of days between today and the delivery date.

Hina: Agreed. Please use our supplied team directory. Accounts can already be created with a setup script. We are not hiring anyone for this delivery cycle. Record developer work only in the estimated hours. We do not need management-hour estimates.

Ayesha: Keep this version simple. We need project details, assigned people, deadlines, and estimated hours. Cost calculation and progress monitoring are outside this challenge.

09:04-09:08 | UrbanCart project scope

Ayesha: First project is UrbanCart Website, for client UrbanCart Clothing. I'll manage it. They need a responsive website where customers can browse products, view product details, and add items to a demo cart. We initially discussed 18 October as the delivery date.

Ali: Do they need a real checkout, payment gateway, and stock integration?

Ayesha: No. For this phase the cart is a demo. Real payments and inventory integration are not included. The client wants to review the buying experience before funding those integrations.

Hamza: So the API scope is product data and a basic cart endpoint, without payment processing?

Ayesha: Correct. Don't add a payment task or an inventory task. The description should make the demo scope clear.

09:08-09:12 | UrbanCart frontend assignment

Ali: I can own the product catalog interface: product listing, a product detail screen, and responsive layout. Put that down as 12 estimated hours, due on 12 October.

Ayesha: Please call that task Product catalog UI. We also need the demo cart interface as a separate task so we can track it separately.

Ali: Yes. Demo cart UI will take 8 hours, due 15 October. That covers adding and removing items, quantities, and a visible total. I am the owner of both frontend tasks.

Bilal: Are these two separate tasks rather than a single 20-hour frontend task?

Ayesha: Exactly. Two tasks, same owner, with the deadlines we just agreed. We need that separation in the task list.

09:12-09:16 | UrbanCart backend and delivery correction

Hamza: For Product and cart APIs, I estimate 14 hours. I own it, and the deadline is 14 October. I will provide product responses and the demo cart endpoints Ali needs.

Ayesha: Good. After that, Ali owns Website integration and testing. Let's start with a six-hour estimate and a 17 October deadline.

Ali: Six hours is reasonable for connecting the screens and checking the demo flow. But please move that task to 19 October. I need a little more calendar space after the API work.

Ayesha: Accepted. Website integration and testing is 6 hours, due 19 October. Also, the client has just confirmed that final project delivery can be 20 October. That replaces the earlier 18 October date. The final UrbanCart project deadline is 20 October.

Hamza: So the final website plan has four tasks, and the new deadline is 20 October. No payment gateway in this phase.

Ayesha: Correct.

09:16-09:20 | QuickServe scope and manager

Bilal: The second project is QuickServe Mobile App, for client QuickServe Services. I am the project manager. The client needs a customer app for signing in, requesting a service, and seeing the request's current status.

Sara: Android only for the demonstration, or do we need separate native apps?

Bilal: A Flutter app demo is enough. We don't need separate Android and iOS development tasks. The project deadline is 24 October.

Usman: What about live maps, driver tracking, and payments?

Bilal: Exclude them. This version is customer login, service booking, and booking status. Those other features may be future work, but they must not appear as tasks in the current project.

09:20-09:24 | QuickServe screen work

Sara: I'll own Login and profile screens. That is 8 hours, due 12 October. It includes the customer login interface and a basic profile screen.

Bilal: Please keep that as one task. We don't need to split every field into its own task.

Sara: The second task is Service booking screens. I'll own that too. I estimate 12 hours, due 17 October. The customer selects a service, enters the request details, and sees a confirmation screen.

Hina: So Sara has two tasks, and both are mobile UI work. The API work is separate.

Bilal: Right. The transcript shouldn't turn these into generic web frontend tasks. This project is the mobile app.

09:24-09:28 | QuickServe API assignment

Hamza: I can build Booking and account APIs for the app. The endpoint scope is basic customer account handling, service requests, and request status. Put me as the owner.

Bilal: What's the effort estimate and delivery date?

Hamza: 16 hours, due 16 October. That's separate from the 14-hour UrbanCart API task. Please don't merge those just because I own both.

Usman: I need those responses for mobile integration, but we can use sample responses while Hamza works.

Bilal: Good. This is still one API task under QuickServe. There is no new shared platform project.

09:28-09:32 | QuickServe integration estimate correction

Usman: I will own Mobile integration and testing. Initially I would put it at 8 hours, due 22 October.

Sara: Can that cover the booking status screen, error states, and testing login through booking? Eight sounds a little tight.

Usman: You're right. Make the final estimate 10 hours. Keep the task deadline at 22 October. I will connect the mobile UI to the API, display request status, and test the whole customer flow.

Bilal: Final agreement: Mobile integration and testing, Usman, 10 hours, 22 October. QuickServe still delivers on 24 October. Don't keep the earlier eight-hour estimate.

Ayesha: That's four tasks for QuickServe as well. We are not adding maps or payment tasks.

09:32-09:36 | HelpDeskPro scope and manager

Hina: Third project is HelpDeskPro AI Assistant, for client HelpDeskPro Solutions. I am managing it. They want a support assistant that answers questions from a supplied FAQ document and passes unresolved questions to a human team.

Zain: Does the assistant need to send emails or connect to a real ticketing service?

Hina: No external message sending is required. Human escalation can be a saved record in the demo. We are building a support proof of concept, not integrating their full support system.

Maryam: We should keep an explicit boundary that the assistant uses the FAQ content instead of guessing unsupported answers.

Hina: Agreed. The project deadline is 22 October. We will test it with some questions that aren't in the document too.

09:36-09:40 | HelpDeskPro document work

Maryam: I'll own FAQ document processing. It should prepare the supplied FAQ so the assistant can retrieve relevant content. I estimate 10 hours, due 13 October.

Hina: Please make the task description clear: prepare and retrieve from the FAQ. Don't make a separate task for every FAQ topic.

Zain: I can then own Assistant answer generation. I'll use the prepared content, connect the model, and handle the response structure.

Hina: Give us the estimate and date for that task.

Zain: 14 hours, due 17 October. If the FAQ doesn't support an answer, the assistant should say it cannot resolve the question rather than inventing a response.

09:40-09:44 | HelpDeskPro escalation

Zain: The next task is Human escalation flow. I can own it as well: save unresolved questions so they can be reviewed by a person. The estimate is 6 hours, due 18 October.

Bilal: Are you assigning those escalated questions to another employee now?

Hina: No, not as new project tasks from this planning meeting. The feature is a saved escalation record in the client's demo. Keep our development task assigned to Zain.

Ayesha: The distinction matters. Discussion of end users should not create employees in our own company directory.

Hina: Exactly. Also, the client mentioned someone called Kamran who may supply a document later. Kamran is not a NovaWorks employee. Do not add him to our team or assign development work to him.

09:44-09:48 | HelpDeskPro testing owner correction

Hina: For Assistant evaluation and testing, I was initially considering Zain as the owner. We need to test FAQ answers, unsupported questions, and the escalation path.

Maryam: I can own that instead. It would be better if someone other than the answer-generation developer checks the results.

Hina: Agreed. Replace the earlier suggestion: Maryam is the final owner of Assistant evaluation and testing.

Maryam: Put the estimate at 8 hours, due 21 October. I'll include normal questions and missing-answer cases. That is separate from my ten-hour FAQ document task.

Hina: Confirmed: Maryam, 8 hours, 21 October. Final HelpDeskPro deadline stays 22 October.

09:48-09:52 | Simple accounts and team setup

Bilal: Please don't spend time building a registration flow. We can use one administrator account, our three manager accounts, and the six developer accounts.

Ayesha: The team names and specializations can be hardcoded or loaded from a setup script. The script may create those users with demo passwords. People should be able to log in using the supplied credentials.

Hina: Agreed. We do not need signup, forgot password, email verification, or a screen for creating and editing users. This is a hackathon demonstration with fictional accounts.

Sara: We still need the existing people available to the AI so it assigns the right names.

Bilal: Exactly. The directory is input to the AI. Projects and tasks should come from the meeting rather than requiring someone to enter all twelve tasks manually.

09:52-09:56 | CRM creation flow

Ayesha: The administrator pastes this transcript inside the CRM and clicks Create from Transcript. A valid result should automatically save all three projects and their tasks.

Hina: If a required person or date cannot be resolved, show a clear message and let the administrator correct it. Do not invent an employee. For this meeting, the final recap supplies all the required information.

Bilal: After creation, show project cards and a project detail screen. Each task needs its owner, deadline, description, and estimated hours. A manager can open their projects; a developer can open their assigned task list.

Usman: Do we need charts, completion percentages, timesheets, or budgets?

Ayesha: No. No cost calculation or progress monitoring. Simple login, project lists, task lists, and transcript automation are enough. Saved projects and tasks should remain after a refresh.

09:56-10:00 | Final recap

Ayesha: Final recap: UrbanCart Website, client UrbanCart Clothing, manager Ayesha, deadline 20 October. Ali owns Product catalog UI: 12 hours, 12 October. Ali owns Demo cart UI: 8 hours, 15 October. Hamza owns Product and cart APIs: 14 hours, 14 October. Ali owns Website integration and testing: 6 hours, 19 October.



Bilal: QuickServe Mobile App, client QuickServe Services, manager Bilal, deadline 24 October. Sara owns Login and profile screens: 8 hours, 12 October. Sara owns Service booking screens: 12 hours, 17 October. Hamza owns Booking and account APIs: 16 hours, 16 October. Usman owns Mobile integration and testing: 10 hours, 22 October.

Hina: HelpDeskPro AI Assistant, client HelpDeskPro Solutions, manager Hina, deadline 22 October. Maryam owns FAQ document processing: 10 hours, 13 October. Zain owns Assistant answer generation: 14 hours, 17 October. Zain owns Human escalation flow: 6 hours, 18 October. Maryam owns Assistant evaluation and testing: 8 hours, 21 October.

Ayesha: Those are the final decisions. Keep the rejected features out. The company already has its nine employees. Create three projects with twelve tasks, then show them in the CRM. That's all for this meeting.

8. Simple Demonstration

1. Run the demo-account setup script or load the fixed demo users; show there is no individual signup requirement.
2. Log in as admin and paste the transcript.
3. Click Create from Transcript and show three saved projects with twelve tasks.
4. Open a project and show its client, manager, deadline, and assigned tasks with estimated hours.
5. Log in as Ayesha and show only UrbanCart. Log in as Ali and show only his three assigned tasks and related project. Verify other users' projects/tasks cannot be accessed.
6. Log in as Hamza and show his two tasks across UrbanCart and QuickServe.
7. Refresh and show saved projects/tasks remain.
8. Try a modified transcript to demonstrate genuine AI conversion rather than a fixed answer.

9. Organizer Verification Reference

Expected output: three projects and twelve tasks. Accounts are already supplied through hardcoding/configuration or a setup script. No user signup, cost calculation, or progress monitoring is evaluated.

Project	Manager	Deadline	Tasks	Hours
UrbanCart Website	Ayesha	2026-10-20	4	40
QuickServe Mobile App	Bilal	2026-10-24	4	46
HelpDeskPro AI Assistant	Hina	2026-10-22	4	38

Hours in this reference let organizers verify extraction; a total-hours dashboard is optional.

Project / task	Owner	Deadline	Hours	Note
UrbanCart / Product catalog UI	Ali	2026-10-12	12	Final
UrbanCart / Demo cart UI	Ali	2026-10-15	8	Final
UrbanCart / Product and cart APIs	Hamza	2026-10-14	14	Final
UrbanCart / Website integration and testing	Ali	2026-10-19	6	Revised date
QuickServe / Login and profile screens	Sara	2026-10-12	8	Final
QuickServe / Service booking screens	Sara	2026-10-17	12	Final
QuickServe / Booking and account APIs	Hamza	2026-10-16	16	Final
QuickServe / Mobile integration and testing	Usman	2026-10-22	10	Revised hours
HelpDeskPro / FAQ document processing	Maryam	2026-10-13	10	Final
HelpDeskPro / Assistant answer generation	Zain	2026-10-17	14	Final
HelpDeskPro / Human escalation flow	Zain	2026-10-18	6	Final
HelpDeskPro / Assistant evaluation and testing	Maryam	2026-10-21	8	Revised owner

Check final decisions: UrbanCart project deadline is 20 October, not 18. Its integration task is due 19 October, not 17. QuickServe integration is 10 hours, not 8. HelpDeskPro testing belongs to Maryam, not Zain. Kamran is not a company employee. Do not generate rejected payment, inventory, maps, or real-email integration tasks.

Suggested changed-input test: change QuickServe integration's final estimate to 12 hours and deadline to 23 October. Its generated task should reflect those values; other tasks should remain unchanged.

This section is an organizer answer key and may be removed from the student handout. Prioritize a usable CRM and working AI-to-record creation flow within three hours.

10. GitHub README - Required for Evaluation

Add a README.md file at the root of your GitHub repository. Judges should be able to understand, run, and test the project without guessing.

Include: project/team name; stack; working features; exact setup and run commands; database setup and seed command; environment-variable names; demo emails/passwords; transcript testing steps; live/demo-video links; deployment platform, database provider, and deployment steps; known limitations.

If using a local database, provide a demo video plus instructions for local setup. If deployed, provide the working application link and demo credentials, and explain how frontend, backend, and database were deployed. GitHub stores source code; hosting a repository alone does not deploy the application/database.

Supply .env.example with placeholders. Keep actual API keys and database passwords out of README and GitHub. Fictional demo-user credentials may be listed. A separate README template is provided; replace its placeholders with your actual commands and links before submission.